

Telnora N-ATLAS Kit: Technical Documentation

Submission: National AI Innovation Challenge 2026, Innovation & Enterprise track, PS1 Developer Infrastructure
Team: Telnora Technologies, Abuja, Nigeria **Repository:** <https://github.com/cutewizzy11/telnora-natlas-kit> **Live demo / playground:** <https://telnora-natlas-kit.vercel.app> **Licence:** MIT (toolkit). N-ATLaS models are under the Open-Source Research and Innovation License.

1. Problem

N-ATLaS is released as open weights. A developer who wants to use it in a web or mobile product still has to work out how to serve it, how to call it, how to handle streaming and retries, how to capture and transcribe voice, and how to build a usable UI. Each team repeats this work. The kit removes that repeated work.

2. What the kit provides

Component	Location	Purpose
TypeScript SDK <code>@telnora/natlas</code>	<code>packages/sdk</code>	<code>chat()</code> , <code>stream()</code> , <code>transcribe()</code> ; retries with backoff, timeouts, abort, typed errors. Zero dependencies.
Python SDK <code>telnora-natlas</code>	<code>packages/python</code>	Same API surface, standard library only.
React components	<code>packages/react</code>	<code><NatlasChat /></code> streaming chat box and <code><VoiceRecorder /></code> that records a voice note and returns a transcript.
Playground	<code>app/playground</code>	Try prompts against any endpoint, use voice input, copy generated SDK code. Includes a clearly labelled demo mode backed by a mock server.
Docs portal	<code>app/docs</code>	Quickstart from zero to a first call.
Deployment recipes	<code>deploy/</code>	Docker Compose for vLLM (LLM) and an OpenAI-compatible ASR gateway for N-ATLAS ASR models.

3. Architecture



The SDK targets the OpenAI-compatible wire format, so it works with the self-hosted setup above and with any official N-ATLAS API that follows that format. The base URL is configurable, so switching from a self-hosted

endpoint to an official one is a one-line change.

4. N-ATLAS integration

- **Language model:** NCAIR1/N-ATLaS (fine-tuned Llama-3 8B) is served with vLLM (`vllm serve NCAIR1/N-ATLaS`). The SDK sends standard chat-completion requests, including `repetition_penalty`, `temperature`, `max_tokens`.
- **Speech recognition:** NCAIR1/NigerianAccentedEnglish (Whisper Small, 244M parameters) is loaded with the Hugging Face `transformers` ASR pipeline behind `deploy/asr_server.py`, which exposes an OpenAI-style `/v1/audio/transcriptions` route. Browser recordings (webm/opus) are decoded with `ffmpeg`. The model's documented limit is 30 seconds per inference; the gateway chunks longer audio.
- **Other ASR checkpoints** (Yoruba-ASR, Hausa-ASR, Igbo-ASR) can be served through the same gateway by setting `ASR_MODEL`.

5. Setup and usage

Serve the models

```
cd deploy
docker compose up --build      # LLM on :8000 (needs an NVIDIA GPU), ASR on :8001
```

TypeScript

```
import { Natlas } from "../natlas"; // copy packages/sdk/src/index.ts
const natlas = new Natlas({ baseUrl: "http://HOST:8000/v1" });
const { text } = await natlas.chat([{ role: "user", content: "Hello" }]);
for await (const chunk of natlas.stream([{ role: "user", content: "Hello" }])) process.stdout.write(chunk);
```

Python

```
from natlas import Natlas
client = Natlas("http://HOST:8000/v1")
print(client.chat([{"role": "user", "content": "Hello"}]))
```

React

```
<NatlasChat client={new Natlas({ baseUrl: LLM_URL })} asrClient={new Natlas({ baseUrl: ASR_URL })} />
```

6. Quality and testing

- TypeScript SDK: 6 unit tests (`npm test` in `packages/sdk`) covering success path, auth header and model, retry on 5xx, no retry on 4xx, SSE streaming across chunk boundaries, multipart transcription, config validation.
- Python SDK: 6 tests (`python -m unittest discover -s tests -t .` in `packages/python`) against a local HTTP server covering the same behaviours.
- The Next.js app type-checks and builds for production; the demo site is deployed on Vercel.

7. Honest status and limitations

- The public demo runs in **demo mode** by default, which returns simulated text from a mock server. It is not model output. Real N-ATLAS output requires the user to point the playground at a hosted endpoint.
- The deployment recipes follow the model cards and vLLM documentation; validation on real hardware is reported in the beta-test evidence and repository issues.
- The 30-second ASR limit and reduced accuracy on noisy audio and code-switching are properties of the underlying model.
- The SDK is currently distributed as source (one file) and via pip from git; publishing to npm and PyPI is planned.

8. Scalability and sustainability

- The SDK is transport-level and stateless, so scale is determined by the model host. Retry and timeout handling make it safe behind load balancers.
- Roadmap: npm and PyPI packages; React Native component; a fine-tuning starter notebook; sample apps (EdTech tutor first, since Telnora serves EdTech clients); community contributions via the repository's CONTRIBUTING guide.
- Telnora will maintain the kit because it is the base for the agency's own AI products for African clients.

9. Licence and attribution

The toolkit is MIT licensed. N-ATLAS is released under the Open-Source Research and Innovation License, which caps deployments at under 1,000 active end-users without a separate licence and requires attribution to Awarri Technologies and the Federal Ministry of Communications, Innovation and Digital Economy. The kit's docs and footer carry this attribution, and the deployment README repeats the cap so downstream developers are aware.